



SQL DML e Consultas

Aula 14 · Bloco 4 — SQL e Projeto Físico

Povoar o banco — e enfim fazer as perguntas

Nesta aula

1. `INSERT`, `UPDATE`, `DELETE`
2. `SELECT`: a estrutura e a **ordem real**
3. **Junções**
4. **Agrupamento** e agregação
5. **Subconsultas** e a `VIEW`

INSERT — e a regra de ouro

```
INSERT INTO obra (titulo, ano, isbn)
VALUES ('Dom Casmurro', 1899, '9788512345678');
```

✏ **Sempre liste as colunas.** `INSERT INTO obra VALUES (...)` depende da **ordem física** — uma ordem que o modelo relacional diz não existir (aula 09). Um `ALTER TABLE ADD COLUMN` quebra todo `INSERT` que confiava nela, **e quebra em silêncio** se os tipos forem compatíveis.

A ordem da carga

Você **não pode** inserir um empréstimo antes do usuário existir — a integridade referencial impede.

- | | |
|-------------------------------|---------------------------------|
| 1. USUARIO, OBRA, AREA, AUTOR | ← as independentes |
| 2. EXEMPLAR | ← depende de OBRA |
| 3. EMPRESTIMO | ← depende de USUARIO e EXEMPLAR |
| 4. MULTA, RENOVACAO | ← dependem de EMPRESTIMO |

Dos pais para os filhos. É o inverso do **DROP**.

O **WHERE** esquecido

```
UPDATE usuario SET curso = 'SI';  
DELETE FROM emprestimo;
```

Os dois comandos são **válidos**.

Os dois atingem **a tabela inteira**.

Hábito que salva: escreva primeiro o **SELECT**
com o mesmo **WHERE**, veja o que ele traz,
e só então troque por **UPDATE** ou **DELETE**.

SELECT : a ordem que você escreve x a ordem que roda

```
SELECT    colunas          -- 5º
FROM      tabelas          -- 1º
WHERE     condição         -- 2º
GROUP BY  colunas          -- 3º
HAVING    condição         -- 4º
ORDER BY  colunas          -- 6º
```

💡 É por isso que você **não pode** usar um apelido do **SELECT** dentro do **WHERE** : quando o **WHERE** roda, o **SELECT** ainda não aconteceu.

Junções

```
SELECT u.nome, o.titulo
FROM emprestimo e
JOIN usuario u ON u.matricula = e.matricula
JOIN exemplar x ON x.cod_obra = e.cod_obra
JOIN obra o ON o.cod_obra = x.cod_obra;
```

INNER JOIN — só o que casa dos dois lados.

LEFT JOIN — tudo da esquerda, mesmo sem par.



LEFT JOIN ... WHERE direita IS NULL é como se responde "obras que **nunca** foram emprestadas".

Agrupamento e agregação

```
SELECT    u.curso, COUNT(*) AS total
FROM      emprestimo e
JOIN      usuario u ON u.matricula = e.matricula
GROUP BY  u.curso
HAVING    COUNT(*) > 10
ORDER BY  total DESC;
```

WHERE filtra linhas, antes de agrupar. **HAVING** filtra grupos, depois.

⚠ **column must appear in the GROUP BY clause** : toda coluna do **SELECT** fora de uma função de agregação **precisa** estar no **GROUP BY** .

Subconsultas

```
-- IN: existe na lista?  
SELECT nome FROM usuario  
WHERE matricula IN (SELECT matricula FROM emprestimo);  
  
-- EXISTS: existe alguma linha? (costuma ser mais rápido)  
SELECT nome FROM usuario u  
WHERE EXISTS (SELECT 1 FROM emprestimo e WHERE e.matricula = u.matricula);
```

E a **divisão relacional** da aula 11 vira dupla negação: **NOT EXISTS (... NOT EXISTS ...)**.

VIEW : o nível externo, enfim

```
CREATE VIEW emprestimos_em_aberto AS
SELECT u.nome, o.titulo, e.data_prevista
FROM emprestimo e
JOIN usuario u ON u.matricula = e.matricula
JOIN obra o ON o.cod_obra = e.cod_obra
WHERE e.data_devolucao IS NULL;
```

Lembra a aula 02? **Cada visão externa é uma VIEW**. O ciclo fecha aqui.

Exercícios da aula

Na pasta `aula-14/` :

1. `ex01.sql` — a carga completa, na ordem certa, com as colunas listadas;
2. `ex02.sql` — cinco consultas com junção múltipla;
3. `ex03.sql` — três agregações com `GROUP BY` e `HAVING` ;
4. `ex04.sql` — a mesma pergunta com `IN` e com `EXISTS` , comparando;
5. **Desafio** 🌶️ `ex05.sql` — uma divisão relacional, e uma `VIEW` para cada perfil de usuário.

Próxima aula

Aula 15 — Projeto Físico e Transações

O que acontece embaixo — índices,
ACID e concorrência.